

未知环境自主探索规划技术文档

快速部署

依赖安装

本项目基于 ROS Noetic 在 Ubuntu 20.04 系统上开发，如需部署，请先安装 ROS 及相关依赖。安装方法请查阅 [ROS 网站](#)。

若要编译本项目，还需要安装如下依赖

- PCL $\geq$ 1.6，请查阅 [PCL 网站](#).
- Eigen $\geq$ 3.3.4，请查阅 [Eigen 网站](#).
- glfw3，用于仿真器渲染，详细了解请查阅 [GLFW 网站](#)
- ZeroMQ，用于多机通信的消息协议，详细了解请查阅 [ZeroMQ 网站](#)
- NLOpt，用于轨迹优化，详细了解请查阅[文档](#)
- LKH，用于多机任务分配，详细了解请查阅[文档](#)

您可以选择运行依赖自动安装脚本或手动安装

- 运行自动安装脚本

```
1 | ./install_dependencies.sh
```

- 手动安装
 - 安装仿真渲染和代码编译依赖

```
1 | sudo apt-get install -y build-essential libglfw3-dev libglew-dev libzmqpp-dev libdw-dev
```

- 安装 ROS 依赖

```
1 | sudo apt-get install -y ros-noetic-cv-bridge ros-  
noetic-tf2-ros ros-noetic-tf2-geometry-msgs ros-  
noetic-tf2-eigen ros-noetic-tf2
```

- 编译安装 NLOpt

```
1 | git clone git@github.com:stevengj/nlopt.git  
2 | cd nlopt  
3 | mkdir build && cd build  
4 | cmake .. && make -j$(nproc)  
5 | sudo make install
```

- 编译安装 LKH

```
1 | wget http://akira.ruc.dk/~keld/research/LKH-  
3/LKH-3.0.6.tgz  
2 | tar xvfz LKH-3.0.6.tgz  
3 | cd LKH-3.0.6  
4 | make -j$(nproc)  
5 | sudo cp LKH /usr/local/bin
```

编译安装本项目

```
1 | mkdir -p exploration_ws/src  
2 | cd exploration_ws/src
```

将本项目的代码拷贝到该目录下，并使用 catkin_make 编译本项目

```
1 | cd ..  
2 | source /opt/ros/noetic/setup.sh  
3 | catkin_make
```

半仿真测试

如果要进行多机半仿真测试，可以通过如下方法启动代码。

假设有 5 架飞机参与仿真，每架飞机通过网线与主机连线。主机上运行仿真器，每架飞机上运行探索和控制代码。

请预先配置多机网络通信保证每架飞机的 ROS 消息可以相互连通。

话题配置

启动规划器时，每架无人机都需要在

`swarm_exploration/exploration_manager/launch/run_in_half_sim.launch` 文件中配置如下参数，以保证飞机能收到仿真器发来的图像、点云和全局位姿信息：

```
1 <arg name="odometry_topic" default="lidar_slam/odom"/>
2 <arg name="sensor_pose_topic"
  default="pcl_render_node/sensor_pose"/> <!-- 传感器在世界系中的位姿 -->
3 <arg name="depth_topic"
  default="pcl_render_node/depth"/> <!-- 深度图或点云 -->
4 <arg name="cloud_topic"
  default="pcl_render_node/cloud"/> <!-- 深度图或点云 -->
```

飞机通过 `px4ctrl` ROS 节点将轨迹点转为 `mavros` 控制指令通过 `mavlink` 与飞控通信。请启动 `mavros` 并且正确配置通信串口和波特率。运行过程中，该 ROS 节点可以将飞机切换到 `OFFBOARD` 状态并自动起飞。相关代码在 `./px4ctrl` 文件夹中。您也可以使用其他控制器实现飞机轨迹点到 `mavros` 控制指令的转化。

启动调试

1. 在主机上启动地面站。以下代码会打开一个只有坐标轴的 RVIZ 窗口。

```
1 source devel/setup.sh
2 roslaunch exploration_manager ground_station.launch
  drone_num:=5
```

1. 每架飞机分别启动规划器，启动时需要设置不同的 `drone_id`，且不要超过 `drone_num`

```
1 source devel/setup.sh
2 roslaunch exploration_manager run_in_half_sim.launch
  drone_id:=1 drone_num:=5
```

2. 您可以在打开的 RVIZ 窗口中看到 5 架飞机依次出现，然后出现彩色的不透明点云地图（代表地面站收到的融合后的地图）。地面上的彩色半透明区块代表边界区域，该部分会随着飞机探索逐渐消失。
3. 如果以上部分正常，则可以用如下命令给 `px4ctrl` 节点发送飞机自动起飞指令

```
1 rostopic pub -1 /px4ctrl/takeoff_land
  quadrotor_msgs/TakeoffLand "takeoff_land_cmd: 1"
```

4. 飞机自动起飞后，使用 RVIZ 中的 2D Nav Goal 工具向飞机发送探索启动指令，即可以看到 5 架飞机向各个方向散开进行协同探索。

全仿真测试

使用代码中实现的轻量仿真器。

1. 切换到 `exploration_ws` 目录下，使用 `source devel/setup.sh` 刷新该项目的 ROS 环境变量

2. 启动可视化

```
1 source devel/setup.sh
2 roslaunch exploration_manager rviz_sim.launch
```

3. 在相同位置打开一个终端，启动规划器和仿真器

```
1 source devel/setup.sh
2 roslaunch exploration_manager
  swarm_exploration_sim.launch
```

4. 您可以在打开的 RVIZ 窗口中看到 5 架飞机依次出现，然后出现灰色的点云地图（代表未探索区域）和彩色的不透明点云（代表已经探索的区域）。地面上的彩色半透明区块代表边界区域，该部分会随着飞机探索逐渐消失。
5. 点击 RVIZ 中的 2D Nav Goal，再点击区域内一点，向飞机发送探索启动指令。你可以看到 5 架飞机向各个方向散开开始协同探索。

修改和迁移

如果需要修改相关配置，需要修改如下文件中的配置：

```
swarm_exploration/exploration_manager/launch/project/swarm_exploration_sim.launch
```

该文件中分别启动了每架飞机的仿真器和规划器。

仿真器的启动 launch 文件为

```
swarm_exploration/exploration_manager/launch/marsim_single_drone.xml
```

规划器的启动 launch 文件为

```
swarm_exploration/exploration_manager/launch/run_in_half_sim.launch
```

规划器

基本设置

规划器部分在 RACER 基础上根据项目需求进行了相关开发。基本参数可以在

```
swarm_exploration/exploration_manager/launch/run_in_half_sim.launch
```

 中进行设置。

该文件包含两个节点，`exploration_node` 探索节点规划出一条用B样条参数化的安全无碰撞光滑轨迹；`traj_server` 轨迹节点把参数轨迹转化成固定频率的目标点，发送给仿真器或者控制器（实机实验）。

启动探索节点需要设置的参数如下：

```
1 <arg name="drone_id" default="1"/>           <!-- 该飞机在多机系统中的编号 -->
2 <arg name="drone_num" default="4"/>          <!-- 多机系统中飞机的个数 -->
3 <arg name="init_x" default="0"/>             <!-- 世界系中初始位姿 -->
4 <arg name="init_y" default="0"/>
5 <arg name="init_z" default="0"/>
6 <arg name="init_yaw" default="0.0"/>
7 <arg name="map_size_x" default="35"/>        <!-- 探索区域地图大小（多个飞机必须一致） -->
8 <arg name="map_size_y" default="35"/>
9 <arg name="map_size_z" default="3.5"/>
10 <arg name="simulation" default="false"/>
11 <arg name="odometry_topic" default="lidar_slam/odom"/>
12 <arg name="sensor_pose_topic"
    default="pcl_render_node/sensor_pose"/> <!-- 传感器在世界系中的位姿 -->
13 <arg name="depth_topic"
    default="pcl_render_node/depth"/> <!-- 深度图或点云 -->
14 <arg name="cloud_topic"
    default="pcl_render_node/cloud"/> <!-- 深度图或点云 -->
15 <arg name="cx" default="324.0879821777344"/>
    <!-- 相机外参 -->
16 <arg name="cy" default="239.10362243652344"/>
17 <arg name="fx" default="385.69793701171875"/>
18 <arg name="fy" default="385.69793701171875"/>
```

探索节点需要正确配置如下消息

- 里程计 `odometry_topic`
- 传感器位姿 `sensor_pose_topic` （可以和 `odometry_topic` 设置的一致）
- 深度图或者点云 `depth_topic` `cloud_topic` （相机用深度图，LiDAR 用点云，若用深度图需要写对相机外参）

探索节点和轨迹节点中通过 `/planning/bspline` 消息耦合。轨迹节点以固定频率发布控制指令 `/position_cmd`。该消息定义在 `Utils/mars_quadrotor_msgs/msg/PositionCommand.msg`，其中包含了轨迹状态以及飞机的目标点、目标速度、目标加速度等状态变量。

```
1 Header header
2 geometry_msgs/Point position
3 geometry_msgs/Vector3 velocity
4 geometry_msgs/Vector3 acceleration
5 geometry_msgs/Vector3 jerk
6 geometry_msgs/Vector3 angular_velocity
7 geometry_msgs/Vector3 attitude
8 geometry_msgs/Vector3 thrust
9 float64 yaw
10 float64 yaw_dot
11 float64 vel_norm
12 float64 acc_norm
13
14
15 float64[3] kx
16 float64[3] kv
17 uint32 trajectory_id
18 uint8 TRAJECTORY_STATUS_EMPTY = 0
19 uint8 TRAJECTORY_STATUS_EMER = 2
```

```
20 uint8 TRAJECTORY_STATUS_READY = 1
21 uint8 TRAJECTORY_STATUS_COMPLETED = 3
22 uint8 TRAJECTORY_STATUS_ABORT = 4
23 uint8 TRAJECTORY_STATUS_ILLEGAL_START = 5
24 uint8 TRAJECTORY_STATUS_ILLEGAL_FINAL = 6
25 uint8 TRAJECTORY_STATUS_IMPOSSIBLE = 7
26 uint32 ACTION_STOP = 8
27 # Its ID number will start from 1, allowing you
   comparing it with 0.
28 uint8 trajectory_flag
```

进阶设置

关于探索部分的更加精细的配置，可以通过修改

```
swarm_exploration/exploration_manager/launch/planner_sim.xml
```

1 中对应参数实现。

快速探索需要通过边界信息结构在整个空间汇总维护探索规划所需的关键信息。边界只在给定的矩形区域内生成。如下参数设置了矩形区域的范围。

```

1 <param name="sdf_map/box_min_x" value="-4.25"
  type="double"/>
2 <param name="sdf_map/box_min_y" value="-7.5"
  type="double"/>
3 <param name="sdf_map/box_min_z" value="0"
  type="double"/>
4 <param name="sdf_map/box_max_x" value="4.25"
  type="double"/>
5 <param name="sdf_map/box_max_y" value="7.5"
  type="double"/>
6 <param name="sdf_map/box_max_z" value="2"
  type="double"/>

```

在多机协同探索中多个无人机需要相互通信实现任务分配、地图融合和避撞。这部分通信依赖于如下消息

```

1 <!-- 探索任务分配 -->
2 <remap from="/swarm_expl/drone_state_send"
  to="/swarm_expl/drone_state" />
3 <remap from="/swarm_expl/drone_state_recv"
  to="/swarm_expl/drone_state" />
4 <remap from="/swarm_expl/pair_opt_send"
  to="/swarm_expl/pair_opt" />
5 <remap from="/swarm_expl/pair_opt_recv"
  to="/swarm_expl/pair_opt" />
6 <remap from="/swarm_expl/pair_opt_res_send"
  to="/swarm_expl/pair_opt_res" />
7 <remap from="/swarm_expl/pair_opt_res_recv"
  to="/swarm_expl/pair_opt_res" />
8 <remap from="/swarm_expl/grid_tour_send"
  to="/swarm_expl/grid_tour" />
9 <remap from="/swarm_expl/hgrid_send"
  to="/swarm_expl/hgrid" />
10 <!-- 多机地图融合 -->

```

```
11 <remap from="/multi_map_manager/chunk_stamps_send"
    to="/multi_map_manager/chunk_stamps" />
12 <remap from="/multi_map_manager/chunk_data_send"
    to="/multi_map_manager/chunk_data" />
13 <remap from="/multi_map_manager/chunk_stamps_recv"
    to="/multi_map_manager/chunk_stamps" />
14 <remap from="/multi_map_manager/chunk_data_recv"
    to="/multi_map_manager/chunk_data" />
15 <!-- 多机避撞 -->
16 <remap from="/planning/swarm_traj_recv"
    to="/planning/swarm_traj" />
17 <remap from="/planning/swarm_traj_send"
    to="/planning/swarm_traj" />
```

仿真器

地图设置

该仿真器中通过 `swarm_exploration_sim.launch` 中的 `map_name` 的参数指定地图的 `.pcd` 文件，通过 `map_pub` 节点读取并发布为 `/map_generator/global_cloud` 点云消息。读取到的地图范围通过 `map_size_x`, `map_size_y`, `map_size_z` 参数给定，并通过 `downsample_resolution` 确定降采样后的点云地图分辨率。

仿真器配置

仿真器的感知和动力学部分通过 `marsim_single_drone.xml` 实现。该文件引用 `test_interface/launch/single_drone_racer_project.xml` 中启动的若干节点，实现对于飞机动力学和点云感知的高精度高保真仿真。

为了保证仿真器正常运行，需要给定以下参数：

```
1 <include file="$(find
test_interface)/launch/single_drone_racer_project.xml"
>
2 <arg name="drone_id" value="$(arg drone_id)"/>
3 <arg name="uav_num_" value="$(arg drone_num)"/>
4 <arg name="sensor_type" value="mid360"/>
5 <arg name="init_x_" value="$(arg init_x)"/>
6 <arg name="init_y_" value="$(arg init_y)"/>
7 <arg name="init_z_" value="$(arg init_z)"/>
8 <arg name="init_yaw" value="$(arg init_yaw)"/>
9 <arg name="map_name_" value="$(arg map_name)"/>
10 <arg name="downsample_resolution_" value="$(arg
downsample_resolution)"/>
11 <arg name="odom_topic" value="lidar_slam/odom"/>
12 <arg name="use_gpu" value="$(arg use_gpu)"/>
13 <arg name="use_uav_extra_model_" value="$(arg
use_uav_extra_model)"/>
14 </include>
```

其中 `sensor_type` 为 `mid360` 时代表飞机使用 Mid-360 LiDAR 传感器进行仿真，为 `D435i` 时代表飞机使用 Realsense 相机进行仿真。